

---

```
File "abcTraceback.py", line 3, in a
    b() # Call b().
File "abcTraceback.py", line 7, in b
    c() # Call c().
```

---

We can tell from the `line 3, in a` text that `b()` was called on line 3 inside the `a()` function, which led to `c()` being called on line 7 inside the `b()` function. Notice that the `print()` calls on lines 2, 6, and 10 aren't displayed in the traceback, even though they ran before the function calls occurred. Only the lines containing function calls that lead up to the exception are displayed in the traceback.

The last frame summary shows the line that caused the unhandled exception, followed by the name of the exception and the exception's message:

---

```
File "abcTraceback.py", line 11, in c
    42 / 0 # This will cause a zero divide error.
ZeroDivisionError: division by zero
```

---

Note that the line number given by the traceback is where Python finally detected an error. The true source of the bug could be somewhere before this line.

Error messages are notoriously short and inscrutable: the three words `division by zero` won't mean anything to you unless you know that dividing a number by zero is mathematically impossible and a common software bug. In this program, the bug isn't too hard to find. Looking at the line of code in the frame summary, it's clear where in the `42 / 0` code the zero divide error is happening.

But let's look at a more difficult case. Enter the following code into a text editor and save it as *zeroDivideTraceback.py*:

---

```
def spam(number1, number2):
    return number1 / (number2 - 42)

spam(101, 42)
```

---

When you run this program, the output should look like this: